

Project 4: Insurance Authorization Co-pilot

Given a **CPT**, **payer**, and **plan tier** (HMO/PPO/EPO/MA), the crew determines “**Is prior authorization required?**” with **evidence** and a short rationale, then (optionally) routes to a **human approver** before replying.

Tech Stack

- **CrewAI** (multi-agent orchestration)
 - **Tools**
 - **Serper** web search (free tier) to find payer policy pages (for freshness signals)
 - **Local RAG** (FAISS/Chroma) over project2/docs/payer_policies/*.html
 - **Policy Parser** (regex + LLM wrapper) to extract PA rules per CPT & plan tier
 - **Validation tool** to enforce output schema & citations
 - **Eval & Monitoring: Opik or Langfuse**
 - **Data** (already created in your sandbox)
 - Policies: project2/docs/payer_policies/*.html
 - Probes: project2/data/examples.csv
-

Crew Design

Roles (Agents)

1. **Intent Router**
 - Classifies user query type (PA required? coverage limits? exclusions?)
 - For this project, keep scope to “Is PA required?” and normalize inputs.
2. **Policy Searcher**
 - Uses **Serper** to find the latest payer policy URLs and record them as *external references*.
 - De-duplicates against local policy filenames (helps simulate freshness vs local ground truth).
3. **Policy Retriever (RAG)**
 - Vectorsdb over docs/payer_policies/
 - Retrieves the most relevant passages for the CPT + payer + plan tier; returns snippets + source file.
4. **Policy Parser**
 - Extracts **modality/CPT ranges, plan-tier rules, exceptions**.

- Implements a small **regex/LLM hybrid**: regex for explicit CPTs (70450, 72148, 93306...), LLM for ranged language (“70551–70553”).

5. Decision Synthesizer

- Produces **final JSON**:

```
{
  "cpt": "72148",
  "payer": "Acme Health",
  "plan_tier": "HMO",
  "pa_required": "Y",
  "rationale": "...",
  "evidence": [
    {"type": "local", "file": "acme_imaging.html", "section": "MRI", "span": "..."},
    {"type": "web", "url": "https://...", "note": "policy page found via Serper"}
  ]
}
```

- Must cite at least one **local** source; web evidence is optional/secondary.

6. Compliance Reviewer (optional but recommended)

- Ensures no PHI, verifies citations exist, enforces schema (via Pydantic/JSON Schema validator tool).
- Can block/return to Synthesizer with fix-ups.

7. (Optional) Human Approver

- Simple CLI “approve/reject/edit rationale” gate before the final response.

File/Repo Structure

crew-pa-copilot/

.env.example

requirements.txt

README.md

data/

examples.csv # already generated

docs/

payer_policies/*.html # already generated

```
src/
  crew_app.py          # CLI entry
  agents/
    router.py
    searcher.py
    retriever.py
    parser.py
    synthesizer.py
    compliance.py
  tools/
    serper_tool.py
    retriever_tool.py   # FAISS/Chroma wrapper
    parser_tool.py      # regex + LLM helpers
    schema_validator.py
    logger.py           # Langfuse/Opik hooks
  eval/
    evaluate.py         # runs on data/examples.csv
    metrics.py          # exactness, evidence overlap, latency
  outputs/
  runs/
  reports/
```

Environment & Dependencies

.env.example

```
AZURE_OPENAI_API_KEY=your_key
SERPER_API_KEY=your_key
LANGFUSE_PUBLIC_KEY=...
LANGFUSE_SECRET_KEY=...
LANGFUSE_HOST=https://cloud.langfuse.com # or your self-host
# OPIK_API_KEY=... # if you choose Opik instead
```

requirements.txt (suggested pins)

```
crewai>=0.51
openai>=1.40
langfuse>=2.36
opik>=0.2.6
faiss-cpu>=1.8.0.post1 # or chromadb>=0.5
beautifulsoup4>=4.12
unstructured>=0.14
python-dotenv>=1.0
pydantic>=2.8
tiktoken>=0.7
requests>=2.32
```

Minimal Wiring (illustrative snippets)

src/crew_app.py

```
import argparse, json, time

from crewai import Agent, Task, Crew

from tools.serper_tool import SerperTool

from tools.retriever_tool import PolicyRetriever

from tools.parser_tool import PolicyParser

from tools.schema_validator import SchemaValidator

from tools.logger import Trace


def build_crew():

    search_tool = SerperTool()

    retriever = PolicyRetriever("docs/payer_policies")

    parser = PolicyParser()

    schema = SchemaValidator()


    router = Agent(

        role="Intent Router",

        goal="Normalize inputs and ensure task is 'prior auth required check!'",
```

```
    backstory="You triage benefits questions.",  
    tools=[], allow_delegation=False, verbose=True  
)
```

```
searcher = Agent(  
    role="Policy Searcher",  
    goal="Find latest payer policy URLs via Serper and record references.",  
    tools=[search_tool], allow_delegation=False, verbose=True  
)
```

```
retr = Agent(  
    role="Policy Retriever",  
    goal="Retrieve local policy passages relevant to CPT/payer/plan tier.",  
    tools=[retriever], allow_delegation=False, verbose=True  
)
```

```
policy_parser = Agent(  
    role="Policy Parser",  
    goal="Extract PA rules for CPT & plan tier from passages/ranges.",  
    tools=[parser], allow_delegation=False, verbose=True  
)
```

```
synthesizer = Agent(  
    role="Decision Synthesizer",  
    goal="Produce final JSON with pa_required Y/N, rationale, citations.",  
    tools=[schema], allow_delegation=False, verbose=True  
)
```

```
compliance = Agent(  
    role="Compliance Reviewer",  
    goal="Enforce schema, ensure citations & no PHI; request fixes if needed.",
```

```

        tools=[schema], allow_delegation=False, verbose=True
    )

    # Crew tasks (high-level)
    t_router = Task(description="Classify & normalize inputs.", agent=router)
    t_search = Task(description="Search web for payer policy refs.", agent=searcher)
    t_retrieve = Task(description="Retrieve local policy passages.", agent=retr)
    t_parse = Task(description="Parse & extract PA rules for CPT/plan.", agent=policy_parser)
    t_synth = Task(description="Synthesize final JSON answer.", agent=synthesizer)
    t_comp = Task(description="Validate JSON, citations, compliance.", agent=compliance)

    return Crew(agents=[router, searcher, retr, policy_parser, synthesizer, compliance],
                tasks=[t_router, t_search, t_retrieve, t_parse, t_synth, t_comp],
                verbose=True)

def run_case(cpt, payer, plan_tier):
    crew = build_crew()
    with Trace(props={"cpt":cpt,"payer":payer,"plan_tier":plan_tier}) as tr:
        result = crew.kickoff(
            inputs={"cpt": cpt, "payer": payer, "plan_tier": plan_tier}
        )
        tr.log_output(result)
    return result

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--cpt", required=True)
    ap.add_argument("--payer", required=True)
    ap.add_argument("--plan_tier", required=True, choices=["HMO","PPO","EPO","MA"])
    args = ap.parse_args()
    out = run_case(args.cpt, args.payer, args.plan_tier)

```

```
print(out)
```

tools/retriever_tool.py (concept)

```
# Wrap FAISS/Chroma vectors over docs/payer_policies/*.html
```

```
# Return: [{"file":"acme_imaging.html","section":"MRI","span":"..."}]
```

tools/parser_tool.py (concept)

```
# Regex for explicit CPTs (e.g., 70450, 72148, 93306)
```

```
# Range handler: parse "70551–70553; 72141–72158" into sets
```

```
# Plan-tier map extraction from table-like HTML
```

tools/schema_validator.py

```
# Pydantic model validates:
```

```
# cpt (str), payer (str), plan_tier (enum), pa_required ("Y"/"N"),
```

```
# rationale (str, <= 1000 chars), evidence (>=1 with local citation)
```

tools/logger.py

```
# Centralize Langfuse/Opik instrumentation (traces, spans, properties, feedback)
```

Running It

Install & setup

```
cd crew-pa-copilot
```

```
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
cp .env.example .env # fill keys
```

Warm up the index (one-time)

```
python -m src.tools.retriever_tool --build-index docs/payer_policies
```

Single query

```
python -m src.crew_app --cpt 72148 --payer "Acme Health" --plan_tier HMO
```

Expected: JSON printed + saved to outputs/runs/run_<timestamp>.json with local citations like acme_imaging.html.

Evaluation & Monitoring

Dataset: data/examples.csv (36 probes)

Metrics

- **Exactness:** final pa_required equals expected_pa_required (Y/N).
- **Evidence Overlap:** simple span-overlap (string contains) vs. labeled policy_file.
- **Latency:** p95 end-to-end time per case; tool call counts.
- **Stability:** retry variance across 3 seeds (optional).

Run eval

```
python -m src.eval.evaluate --dataset data/examples.csv --out outputs/reports/eval.json
```

- Push per-case traces and metrics to **Langfuse** (or **Opik** if you prefer):
 properties={cpt, payer, plan_tier, policy_file}
 metrics={exactness, evidence_overlap, latency_ms}

Langfuse/Opik dashboard

- Create an experiment: crew_pa_copilot_v1
- Compare *with* vs *without* **Policy Searcher** to see impact of external refs.

Team of 4 Split (suggested)

1. **Infra/Indexing:** vectordb, ingestion, Serper key handling, CI
 2. **Parser/Rules:** regex & range parsing; HTML table extraction; edge cases
 3. **Prompts & Synthesis:** agent prompts; JSON schema; rationale quality
 4. **Eval/Observability:** metrics, Langfuse/Opik wiring, latency + cost reporting
-